Cache-Aware Optimization of Matrix Multiplication

Final Paper - Computer Architecture Project

Mirsad Özkan | Student ID: 200029982 | Language: C | Environment: Linux / WSL with GCC

Abstract

This project investigates the effect of memory access patterns on matrix multiplication performance. Two C implementations were compared: a naive version using the i-j-k loop order and an optimized version using the i-k-j loop order. Both implementations perform the same mathematical operation and were tested using identical input matrices by using the same random seed. Experiments were performed for 300x300, 500x500, 700x700, and 1000x1000 matrices. The optimized implementation produced the same sample outputs and achieved lower average execution time for every tested matrix size. The largest improvement was observed for the 1000x1000 case, where average runtime decreased from 3.128471 seconds to 2.404728 seconds, corresponding to a 23.13% improvement.

1. Introduction

Matrix multiplication is a common operation in scientific computing, machine learning, and many systems-level workloads. Although the basic algorithm is simple, the implementation can have a strong effect on performance because modern processors rely heavily on cache memory. When a program accesses memory in a cache-friendly order, it can reduce unnecessary memory access overhead and improve runtime. This project focuses on the hardware-software interface by comparing two different implementations of the same matrix multiplication operation.

2. Project Objective

The objective is to compare a baseline matrix multiplication implementation with an optimized implementation that changes loop order. The project belongs to the optimization category because it takes a baseline systems program and improves its execution time through cache-locality-oriented implementation changes.

3. Implementation Details

Both programs are implemented in C. The matrices are initialized with integer values generated using the same random seed. This ensures that both the naive and optimized versions operate on the same input matrices. The sample output value C[0][0] is printed after multiplication to confirm consistency between the two versions.

Version	Loop Order	Description
Naive	i - j - k	Standard triple nested loop implementation.
Optimized	i - k - j	Changes the memory access order to improve locality.

```
Naive loop structure: for (i) for (j) for (k)
Optimized loop structure: for (i) for (k) for (j)
```

4. Build and Run Procedure

The repository includes a Makefile and README.md so that the benchmark can be reproduced easily. The matrix size can be selected at compile time using the N variable.

```
make clean
make N=1000
make run
```

5. Experimental Setup

The experiments were executed in a Linux / WSL Ubuntu environment using GCC. Each matrix size was tested three times for both implementations. The average execution time was calculated from the three runs. The tested matrix sizes were 300x300, 500x500, 700x700, and 1000x1000. The same random seed was used for both implementations to keep the input matrices consistent.

6. Experimental Results

Matrix Size	Sample Output	Naive Avg (s)	Optimized Avg (s)	Difference (s)	Improvement
300x300	6363	0.066901	0.064853	0.002048	3.06%
500x500	10195	0.282797	0.278188	0.004609	1.63%
700x700	14200	0.947626	0.847426	0.100201	10.57%
1000x1000	20287	3.128471	2.404728	0.723744	23.13%

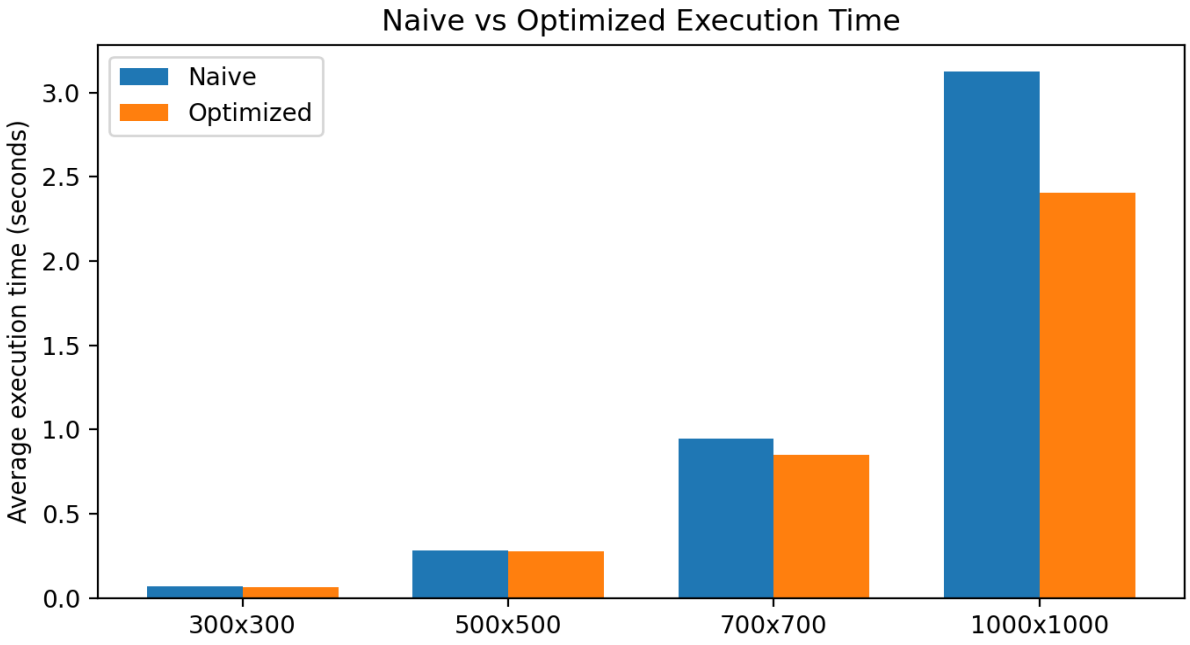

Figure 1. Average execution time comparison for all tested matrix sizes.

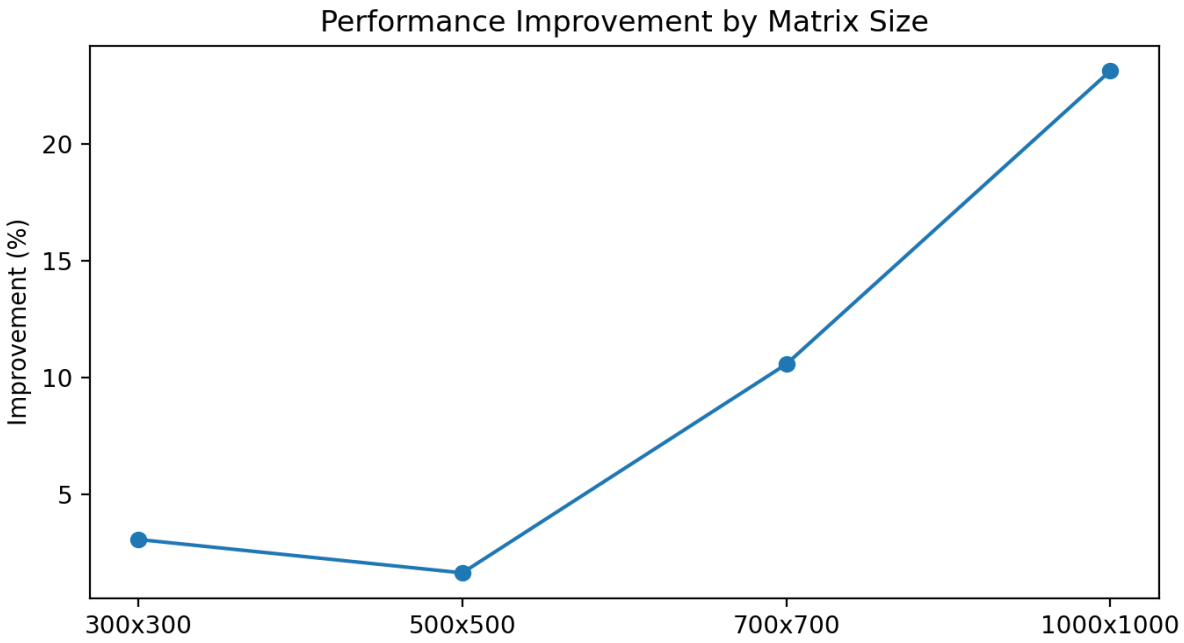

Figure 2. Performance improvement percentage by matrix size.

7. Discussion

The optimized implementation was faster than the naive implementation in every tested case. The improvement is small for 300x300 and 500x500 matrices, but it becomes more visible as the matrix size increases. For 1000x1000 matrices, the optimized loop order improved average execution time by 23.13%. This supports the main idea of the project: two implementations with the same algorithmic complexity can still perform differently because of memory access behavior and cache locality.

8. Repository and Demo

The final repository contains source code, a Makefile, README.md, experimental results, the final paper, and the final presentation. The demo can be reproduced by compiling the project with a selected matrix size and running both implementations. Repository: https://github.com/MrPlasebo/cache-aware-matrix-multiplication/

9. Conclusion

This project demonstrates that implementation details can affect software performance even when the mathematical operation remains unchanged. The optimized matrix multiplication version produced the same sample outputs as the naive version while achieving lower average execution time for all tested matrix sizes. The results show that memory access order is an important factor in performance-sensitive systems programming.